多奇·教育訓練

2026-01-30

使用 GitHub Copilot SDK 打造可自動開發的 AI 代理人

這是一堂會讓你重新校準未來位置的課

多奇數位創意有限公司

技術總監 黃保翕（Will 保哥）

https://blog.miniasp.com

Microsoft® Most Valuable Professional MVP

Experts 2023

認識 GitHub Copilot SDK 框架

GitHub Copilot SDK 是什麼？

- 一套「**讓你把 Copilot 代理 (Agent) 能力嵌入自己應用程式**」的多語言開發工具
- 換句話說，就是把 **Copilot CLI** 直接變成 **軟體開發套件** (SDK) 的玩意！
- Copilot CLI 完整實作了 **ACP** 代理人客戶端協議 (Agent Client Protocol)
  - ACP support in Copilot CLI is now in public preview
- Copilot SDK 目前處於**技術預覽階段** NEW

多奇·教育訓練

代理人客戶端協議 (ACP)

- 代理人用戶端協定（ACP）是一項開放式標準
- ACP 讓任何**代理人**都能**無縫整合**進任何**用戶端的 IDE 編輯環境**
- ACP 基於 JSON-RPC 2.0 規格
- **相關文件**
  - Official ACP documentation
  - Zed — Agent Client Protocol
  - Copilot CLI ACP server
    - copilot --acp --stdio
    - copilot --acp --port 3000
    - 文件內附 ACP 範例程式 👍
  - Zed — Agent Client Protocol
    - 探索不斷成長的 ACP 生態系統
    - Agents on ACP
      - Codex CLI
      - Gemini CLI
      - Claude Code
      - …

多奇·教育訓練

看看 Copilot SDK 的程式碼結構

```
import { CopilotClient } from "@github/copilot-sdk";

const client = new CopilotClient();
await client.start();

const session = await client.createSession({
    model: "gpt-5",
});

await session.send({ prompt: "Hello, world!" });
```

Source

多奇·教育訓練

Copilot SDK 快速上手

環境建置與初始化 (Node.js)

- Copilot SDK 透過 JSON-RPC 以程式化方式控制 GitHub Copilot CLI
- **1 建立資料夾**
  - mkdir copilot-sdk-qs && cd copilot-sdk-qs
- **2 安裝 npm 套件**
  - npm init -y --init-type module
  - npm install @github/copilot-sdk
- **3 安裝 Copilot SDK 技能**
  - npx skills add doggy8088/agent-skills --skill "copilot-sdk" --agent github-copilot --yes
- **4 建立本機 Git 版控（`npx @willh/git-setup`）**
  - git ignore node > .gitignore && git iac

多奇·教育訓練

Copilot SDK 火力展示

- **下載原始碼**
  - `git clone https://github.com/doggy8088/copilot-sdk-demo.git`
  - `cd "copilot-sdk-demo"`
- **範例 1：預設使用 GitHub Copilot 訂閱臉書撰文神器（gpt-5-mini）**
  - `git checkout v1 && npm install && npm start`
- **範例 2：使用 Azure OpenAI 金鑰（BYOK）（查看執行時所有事件）**
  - `git checkout byok && npm install && npm start`
- **範例 3：整合 Gemini CLI 執行（任何支援 ACP 的 Agent 都能跑）**
  - `git checkout gemini && npm install && npm start`
- **範例 4：連接遠端 Copilot CLI 實例**
  - `copilot --acp --port 17817`
  - `git checkout remote && npm install && npm start`

多奇·教育訓練

Copilot SDK 快速上手

SDK	Location	Installation
Node.js / TypeScript	cookbook/nodejs/	`npm install @github/copilot-sdk`
Python	cookbook/python/	`pip install github-copilot-sdk`
Go	cookbook/go/	`go get github.com/github/copilot-sdk/go`
.NET	cookbook/dotnet/	`dotnet add package GitHub.Copilot.SDK`

多奇·教育訓練

取得範例與快速上手 (Node.js)

- **取得專案原始碼**
  - `git clone https://github.com/github/copilot-sdk.git`
  - `cd copilot-sdk`
- **初始化執行環境**
  - `cd cookbook/nodejs/recipe`
  - `npm install`
  - `npx tsx <filename>.ts`
- **更多範例**
  - https://github.com/github/copilot-sdk/tree/main/cookbook
  - 目前支援 .NET, Node.js, Python, Go 等四種語言/框架

多奇·教育訓練

開發你的第一個 Copilot SDK

我想要用 Copilot SDK for Node.js/TypeScript 設計一個代理人：

1. 用戶提供網址
2. 可以自動下載網頁
3. 分析網頁 HTML 並取得主要內容
4. 將主要內容先進行總結
5. 用 zh-tw 寫 3 份臉書貼文（專業新聞報導、輕鬆搞笑文筆、朋友聊天語氣）
6. 將 3 份臉書貼文翻譯為英文

注意：留意非同步事件的 async/await 處理，任務完成請再次檢視有無任何瑕疵！

多奇·教育訓練

Copilot SDK 的設計哲學與系統邊界

不只是 API，而是完整的 Agent Runtime

- **核心定義**
  - 程式化存取 GitHub Copilot CLI 的核心引擎
- **解決痛點**
  - 不需要自己刻寫 **ReAct (Reasoning + Acting) 迴圈**
  - 不需要自己處理 **Context Window** 管理（**記憶壓縮**）
  - 直接沿用 Copilot 的**規劃** (Planning) 與 **工具調用** (Tooling) 能力
- **適用場景**
  - 打造專屬的 CLI 助手
  - 在 IDE 或 GUI 應用中嵌入 AI 代理人
  - CI/CD 自動化腳本（如：自動 Code Review 或自動撰寫單元測試）

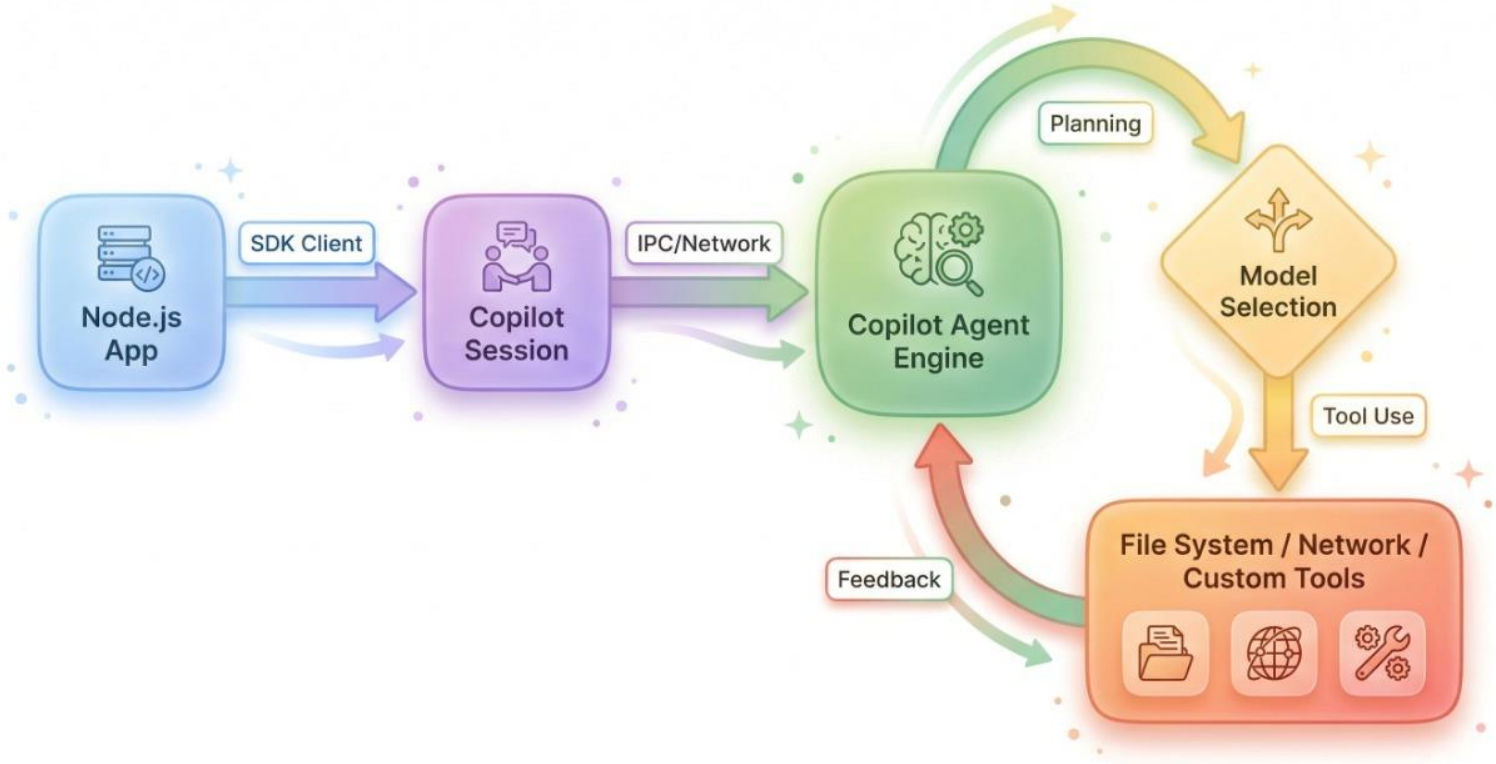
多奇·教育訓練

核心架構元件

元件	說明	關鍵職責
CopilotClient	SDK 入口點	管理與 Copilot CLI Server 的連線、生命週期
Session	對話狀態機	管理多輪對話歷史 (History)、上下文 (Context)、模型參數
Agent	決策大腦	負責接收 Prompt，生成 Plan，並決定是否呼叫工具
Skills (Tools)	手腳	定義 AI 可以執行的具體函式 (Function Calling)
Events	通訊機制	透過 session.start, assistant.*, tool.*, subagent.*, ... 等事件串流回傳結果

多奇·教育訓練

Copilot SDK 運作流程



多奇·教育訓練

Paradigm Shift

從補全到代理的範式轉移

從「補全工具」到「代理人」的關鍵轉換點

- 從「**預測下一步**」到「**直接達成目標**」

- **補全工具**
  - 基於統計的次標記預測 (Next-token Prediction)
  - 使用者必須主動觸發並負責驗證與拼湊

- **代理人**
  - 基於目標的**自主規劃** (Goal-oriented Planning)
  - 代理人負責**調度工具**、**觀察結果**並**自我修正**，直到達成最終目標 🥇

多奇·教育訓練

AI 工具的四大技術轉換點

維度	補全工具 (GitHub Copilot 1.0)	代理人 (GitHub Copilot SDK)
互動模式	單向請求/回應 (Request/Response)	自主循環 (Agentic Loop)
上下文	當前編輯檔案與鄰近代碼	全域上下文 (MCP & Skills & Tool Call)
操作範圍	僅限編輯器內的文本生成	跨系統操作 (Shell, Git, API, DB)
品質保障	人工肉眼檢查	回饋迴圈 (Plan-Execute-Refine)

多奇·教育訓練

工具調用 (Tool Calling) 的引入

- **補全**
  - 模型只會「說」（提供建議），不會「跑」（執行任務）

- **代理**
  - 透過 functions 或 tools 定義，代理可以執行任意命令獲取豐富上下文
  - 例如：執行 npm test、gh pr create 或查詢資料庫

- **架構含義**
  - 模型不再是輸出終點，而是控制中心的邏輯大腦！

多奇·教育訓練

從長上下文到動態上下文 (MCP & Skills)

- **補全**
  - 依賴有限的上下文窗口 (Window Size)

- **代理**
  - 上下文並非多多益善
  - 巧妙利用 Model Context Protocol (MCP)
  - 代理根據任務，動態地從文件、Issue、日誌中提取「剛剛好」的資訊
- **架構含義**
  - 解決了 Context Stuffing (脈絡填充) 造成的幻覺與延遲問題

多奇·教育訓練

計畫與反思 (Reasoning & Refinement)

- **補全**
  - 一次性生成，若錯誤則需人工修改

- **代理**
  - 在 SDK 內部實作 **Thought** -> **Action** -> **Observation** -> **Refine** 迴圈

    思考 -> **行動** -> **觀察** -> **改進**

- **架構含義**
  - 允許模型在報錯時自動嘗試修復 (Self-healing)

多奇·教育訓練

可長時間運作 Agent 的核心設計技巧

長時間運作的 Agent 所帶來的核心挑戰

類型	問題	可能的解決方案
狀態管理	記憶膨脹、上下文混亂	Copilot CLI 正在實作無限記憶機制
流程控制	無限迴圈、重複行為	Copilot CLI 內建無窮迴圈偵測
成本失控	Token 成本失控	多賺點 XD
維運問題	無法 debug、無法重啟	需要經驗
穩定性	API error、rate limit	無解

多奇·教育訓練

什麼是「可長時間運作」Agent

- 讓 Agent 能持續跑、不失控、不累積技術債！
- 長時間運作的 Agent 本質是**可恢復的分散式系統**，不是聊天機器人！
- **定義**
  - 運作時間：數小時、數天、甚至常駐
  - 任務型態：非一次性回應，而是反覆決策、等待、執行
  - 狀態可累積、可恢復、可觀測
- **常見場景**
  - PR reviewer / code bot
  - Background automation agent
  - Chat + tool orchestration agent

多奇·教育訓練

設計原則一：狀態 (State) 必須外部化

- **不要**
  - 把所有狀態塞進 prompt
  - 依賴 LLM「自己記得」
- **一定要**
  - 明確拆分：
    - 短期狀態 (short-term context)
    - 長期狀態 (long-term state)
  - 狀態存放在外部系統 (DB / KV / File)，例如：建立 TODO.md 暫存檔
- **實務建議**
  - Prompt 只放「這一輪必要資訊」
  - 狀態結構化(JSON)，不是自然語言

多奇·教育訓練

設計原則二：Agent ≠ 無限 while loop

- **反模式**

```
while(true):
  think
  act
```

- **正確模型**
  - Agent 是「事件驅動」
  - 每次執行都有明確結束點
- **技巧**
  - 一次執行只做一個 decision
  - 下一步由 scheduler / event 觸發

多奇·教育訓練

設計原則三：明確的「執行邊界」

- **每次 Agent run 必須有**
  - 最大步數 (max steps)
  - 最大 token (即便有 1M Tokens 也可以設定上限 200K Tokens)
  - 最大 wall time (時間到了就停止)
- **原因**
  - 防止 hallucination loop (因為幻覺導致的無窮迴圈) (gemini-2.5-pro)
  - 防止工具濫用 (不要超過 32 個工具註冊)
  - 防止成本爆炸 (透過 session.usage_info 計算成本)
- **實務**
  - Hard limit > Soft instruction

多奇·教育訓練

設計原則四：決策與執行分離

- **拆成兩層**
  - 決策層 (Decision layer) (LLM)
  - 執行層 (Execution layer) (code)
- **LLM 負責**
  - 判斷「要不要做」
  - 決定「做哪一類事」
- **Code 負責**
  - 真正執行
  - 驗證參數
  - 回傳結構化結果

多奇·教育訓練

設計原則五：工具結果必須可預期

- **工具輸出設計**
  - 固定 schema
  - 明確 success / failure
  - 不回自然語言描述
- **原因**
  - Agent 需要「判斷」
  - 不是「閱讀作文」
- **範例**

```
{
  "status": "failed",
  "error_code": "RATE_LIMIT",
  "retry_after": 60
}
```


多奇·教育訓練

設計原則六：錯誤是常態，不是例外

- **假設**
  - API 一定會失敗
  - Tool 一定會 timeout
  - LLM 一定會答非所問
- **設計方式**
  - Retry policy (重試次數)
  - Backoff (退避策略)
  - Fallback (設計後路)
- **重點**
  - 即便是 **錯誤流程** (Error flow) 也要設計 prompt 去處理！

多奇·教育訓練

設計原則七：記憶要「可丟棄」

- **不要**
  - 無限累積 conversation history
- **要**
  - 壓縮 (summarize)
  - 刪除 (expire)
  - 升級 (promote to long-term)
- **心法**
  - Memory 是 cache，不是真理

多奇·教育訓練

設計原則八：可觀測性 (Observability)

- **必備**
  - 每次 decision 的 input / output
  - Tool 呼叫紀錄
  - 狀態變化
- **好處**
  - Debug agent 行為
  - 可以回放 (replay)
  - 人類介入

多奇·教育訓練

設計原則九：人類永遠是最後防線

- **提供**
  - Pause / Resume
  - Manual override
  - Approval gate
- **適用情境**
  - 寫 code
  - 刪資源
  - 發 PR / 發佈

多奇·教育訓練

最小可行架構 (MVP)

- **組件**
  - Agent Runner
  - State Store
  - Tool Adapter
  - Scheduler / Event Source
- **原則**
  - 先能重啟
  - 再談智能

多奇·教育訓練

常見失敗模式

- Prompt 越寫越長
- Agent 變成黑盒
- 出事只能重跑
- 成本事後才發現

多奇·教育訓練

用 Copilot CLI 打造會自己寫程式的 Agent

案例分析

支援 Ralph Wiggum loop 的 Copilot CLI

- **Copilot Ralph Node**
  - https://github.com/doggy8088/copilot-ralph
- **使用範例**
  - 指定迭代次數與完成短語

```
@willh/copilot-ralph run --max-iterations 3 "修正登入流程"
```

  - 讀取 Markdown 或文字檔案作為 prompt

```
@willh/copilot-ralph run task.md
```


多奇·教育訓練

Telegram Bot 作為 Agent 前端的設計模式

- **Telegram Copilot Bot**
  https://github.com/doggy8088/telegram-copilot-bot
  這個專案提供一個 Telegram 機器人，讓您可以直接在聊天裡調用 GitHub Copilot CLI。使用者透過選擇工作目錄並輸入提示詞，便能要求 Copilot 撰寫程式、修改檔案或執行指定操作。機器人會即時顯示 Copilot 產生的訊息與工具執行事件。

- **使用範例**
  - npm install
  - npm run build
  - 設定 .env 環境變數
  - npm start

多奇·教育訓練

Primer 分析你的 Repo 並生成記憶檔案

- **Primer**
  - https://github.com/pierceboggan/primer
  - Primer 是一個指令列工具，它能分析你的程式碼庫並生成 **.github/copilot-instructions.md** 檔案，以協助 AI 程式設計助理更好地理解你的專案。它支援單一儲存庫分析、跨組織批次處理，並內建評估框架來衡量指令的有效性。

多奇·教育訓練

GitHub Copilot Office Add-in

- **GitHub Copilot Office Add-in**
  https://copilot-sdk.gh.miniasp.com/labs/copilot-for-office
- **使用範例**
  - 設定 .env 檔案
  - `docker pull willh/copilot-office-addin:latest`
  - `docker run -d --name copilot-office -p 52390:52390 --env-file .env -v /mnt/g/Projects/copilot-office-addin/certs:/app/certs willh/copilot-office-addin:latest`

多奇·教育訓練

相關連結

- GitHub Copilot CLI SDKs (Repo)
- Building Agents with GitHub Copilot SDK: A Practical Guide to Automated Tech Update Tracking
- Let's Build: Copilot SDK Weekend Contest with Prizes

- 學員課前注意事項
- Copilot SDK 筆記本 (NotebookLM)
- Copilot SDK 問答機器人 (ChatGPT)
- Copilot SDK 問答機器人 (Gemini)

- About agentic memory for GitHub Copilot

聯絡資訊

The Will Will Web

網路世界的學習心得與技術分享

http://blog.miniasp.com/

Facebook

Will 保哥的技術交流中心

http://www.facebook.com/will.fans

Twitter

https://twitter.com/Will_Huang

多奇·教育訓練

THANK YOU!

Q&A